

University of Moratuwa

Department of Electronic and Telecommunication Engineering

EN3150

Assignment 03

Resource-Constrained CNN for Edge Image Classification

Group Members

Lavanathan.J	230371R
Panuharan.S	230462X
Peranavan.K	230474K
Rifnaz.K.R.M	230550P

Contents

1	Introduction	1
2	Dataset and Preprocessing	1
3	Model A: Baseline CNN	2
4	Model B: Resource-Constrained CNN	3
5	Optimizer Comparison	3
6	Custom Model Training and Evaluation	4
7	Pretrained Lightweight Models	5
7.1	MobileNetV2	5
7.2	EfficientNetB0	6
8	Final Model Comparison	7
9	Conclusion	8

1 Introduction

This assignment investigates compact convolutional neural networks for image classification under edge-device constraints. The main design objective is to obtain useful five-class classification performance while keeping memory use and computational cost low enough for resource-constrained deployment. The experiments use the DeFungi image dataset from the UCI Machine Learning Repository [1].

The work compares two custom CNNs and two ImageNet-initialized lightweight transfer-learning models. Model A is a conventional baseline CNN with standard convolutions. Model B is a smaller resource-constrained CNN based on depthwise separable convolutions. MobileNetV2 [2] and EfficientNetB0 [3] are then used as pretrained reference models to compare accuracy, memory footprint, and training cost.

2 Dataset and Preprocessing

Owner: Rifnaz.K.R.M – 230550P

The dataset used in this project is DeFungi from the UCI Machine Learning Repository [1]. It contains microscopic fungi images belonging to five classes: H1, H2, H3, H5, and H6. The verified local dataset contains 9,114 usable RGB JPEG images. No unreadable images were found during dataset verification.

Table 1: Verified class distribution.

Class	Images
H1	4,404
H2	2,334
H3	819
H5	818
H6	739
Total	9,114

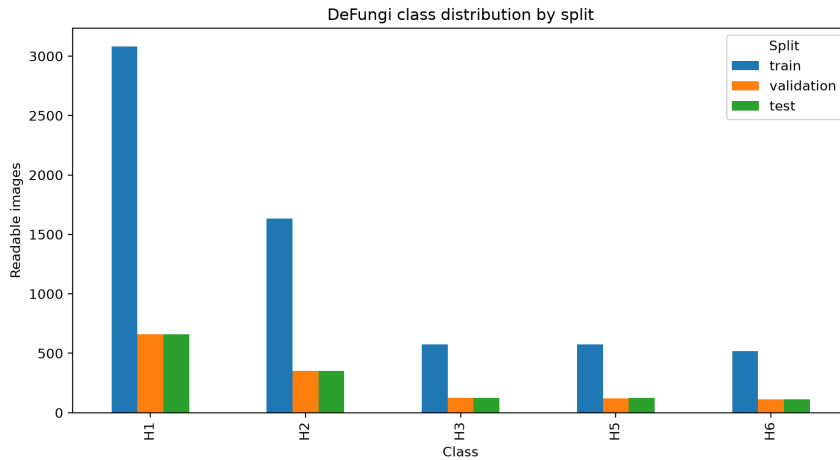


Figure 1: Class distribution of the verified DeFungi dataset.

The data split was created once and reused for all experiments. A stratified 70/15/15 target split was used with seed 42, giving 6,379 training images, 1,367 validation images, and 1,368 test

images. The class membership of the validation and test sets is therefore fixed across custom and pretrained models.

Table 2: Verified stratified split counts.

Split	H1	H2	H3	H5	H6	Total
Train	3,082	1,634	573	573	517	6,379
Validation	661	350	123	122	111	1,367
Test	661	350	123	123	111	1,368

For the custom CNNs, all images were resized to $64 \times 64 \times 3$ and pixel values were normalized to the range $[0, 1]$. For the pretrained models, images were resized to $224 \times 224 \times 3$ and processed with the corresponding Keras application preprocessing functions.

3 Model A: Baseline CNN

Owner: Rifnaz.K.R.M – 230550P

Model A is a conventional CNN baseline using three standard convolutional layers followed by global average pooling and a small dense classifier. ReLU was used after the hidden convolutional and dense layers because it provides a simple non-linearity while avoiding saturation for positive activations. The final Softmax layer produces a probability distribution over the five fungi classes.

Table 3: Model A architecture and parameter count.

Layer	Output Shape	Kernel	Filters/Units	Activation	Parameters
Input	$64 \times 64 \times 3$	–	–	–	0
Conv2D	$64 \times 64 \times 32$	3×3	32	ReLU	896
MaxPool2D	$32 \times 32 \times 32$	2×2	–	–	0
Conv2D	$32 \times 32 \times 64$	3×3	64	ReLU	18,496
MaxPool2D	$16 \times 16 \times 64$	2×2	–	–	0
Conv2D	$16 \times 16 \times 128$	3×3	128	ReLU	73,856
MaxPool2D	$8 \times 8 \times 128$	2×2	–	–	0
GlobalAveragePooling2D	128	–	–	–	0
Dense	64	–	64	ReLU	8,256
Dense	5	–	5	Softmax	325
Total	–	–	–	–	101,829

For a standard convolution layer, the trainable parameter count is

$$P_{\text{conv}} = (K_h K_w C_{\text{in}} + 1) C_{\text{out}}, \quad (1)$$

where the extra 1 accounts for the bias term per output channel. The layer-wise calculations are:

$$\text{Conv1} = (3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896, \quad (2)$$

$$\text{Conv2} = (3 \cdot 3 \cdot 32 + 1) \cdot 64 = 18,496, \quad (3)$$

$$\text{Conv3} = (3 \cdot 3 \cdot 64 + 1) \cdot 128 = 73,856, \quad (4)$$

$$\text{Dense64} = (128 + 1) \cdot 64 = 8,256, \quad (5)$$

$$\text{Dense5} = (64 + 1) \cdot 5 = 325. \quad (6)$$

The total is therefore 101,829 trainable parameters. Global average pooling was used instead of flattening to reduce the classifier size after the convolutional feature extractor.

4 Model B: Resource-Constrained CNN

Owner: Panuharan.S – 230462X

Model B was designed to reduce the parameter count while preserving the same input resolution and output classes as Model A. It replaces standard convolutional layers with depthwise separable convolutions and uses a smaller dense layer.

Table 4: Model B architecture and parameter count.

Layer	Output Shape	Kernel	Filters/Units	Activation	Parameters
Input	$64 \times 64 \times 3$	–	–	–	0
SeparableConv2D	$64 \times 64 \times 32$	3×3	32	ReLU	155
MaxPool2D	$32 \times 32 \times 32$	2×2	–	–	0
SeparableConv2D	$32 \times 32 \times 64$	3×3	64	ReLU	2,400
MaxPool2D	$16 \times 16 \times 64$	2×2	–	–	0
SeparableConv2D	$16 \times 16 \times 96$	3×3	96	ReLU	6,816
MaxPool2D	$8 \times 8 \times 96$	2×2	–	–	0
GlobalAveragePooling2D	96	–	–	–	0
Dense	48	–	48	ReLU	4,656
Dense	5	–	5	Softmax	245
Total	–	–	–	–	14,272

For a depthwise separable convolution with bias, the parameter count is

$$P_{\text{sep}} = K_h K_w C_{\text{in}} + C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \quad (7)$$

The first term is the depthwise spatial convolution, the second term is the pointwise 1×1 convolution, and the final term is the bias. The calculations are:

$$\text{SepConv1} = 3 \cdot 3 \cdot 3 + 3 \cdot 32 + 32 = 155, \quad (8)$$

$$\text{SepConv2} = 3 \cdot 3 \cdot 32 + 32 \cdot 64 + 64 = 2,400, \quad (9)$$

$$\text{SepConv3} = 3 \cdot 3 \cdot 64 + 64 \cdot 96 + 96 = 6,816, \quad (10)$$

$$\text{Dense48} = (96 + 1) \cdot 48 = 4,656, \quad (11)$$

$$\text{Dense5} = (48 + 1) \cdot 5 = 245. \quad (12)$$

Model B has 14,272 trainable parameters, which is below the 100,000-parameter target. Its FP32 parameter storage is 55.75 KB. Compared with Model A, it removes 87,557 parameters, a reduction of about 85.98%.

5 Optimizer Comparison

Owner: Peranavan.K – 230474K

The optimizer comparison used Model A in a controlled limited-training-batch experiment. Each optimizer was trained for 5 epochs using 30 training batches per epoch, while evaluation used the full validation split. The optimizer was selected by the lowest final validation loss, and the selected optimizer was then used for the full custom-model training runs.

Table 5: Controlled optimizer comparison.

Optimizer	Learning Rate	Momentum	Final Val. Accuracy	Final Val. Loss
Adam	0.001	0.0	0.5091	1.2509
SGD	0.010	0.0	0.4835	1.4039
SGD + Momentum	0.010	0.9	0.4835	1.3385

Momentum updates maintain a velocity term:

$$v_t = \beta v_{t-1} + g_t, \quad (13)$$

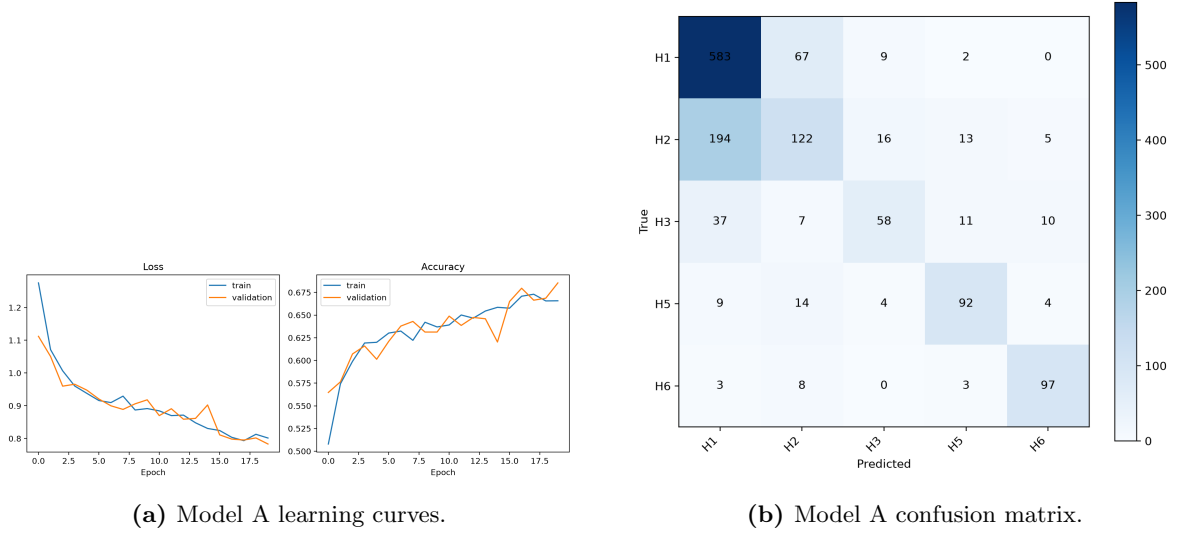
$$\theta_t = \theta_{t-1} - \eta v_t, \quad (14)$$

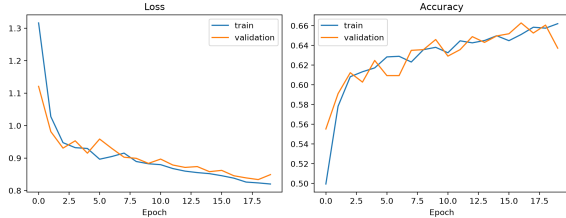
where g_t is the current gradient, η is the learning rate, and β is the momentum coefficient. In this controlled comparison, Adam with learning rate 0.001 achieved the lowest final validation loss of 1.2508898973464966. These values are optimizer-selection evidence only and are not the final test metrics.

6 Custom Model Training and Evaluation

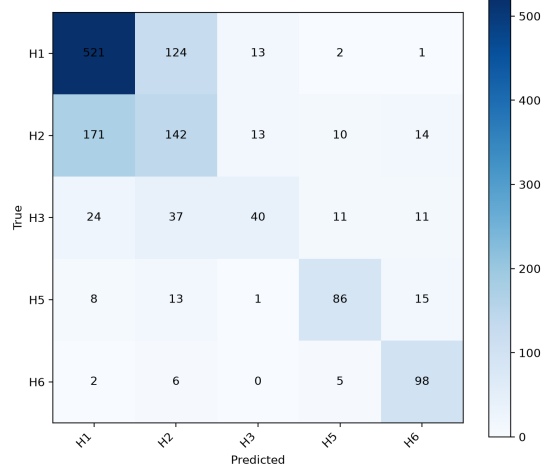
Owner: Peranavan.K – 230474K

Both custom CNNs were trained using the same split and training configuration: 20 epochs, batch size 32, Adam optimizer with learning rate 0.001, and sparse categorical crossentropy. The experiments were run in a CPU-only TensorFlow environment. Test-set evaluation reports accuracy, macro precision, macro recall, macro F1-score, and confusion matrices.

**Figure 2:** Model A training and test evaluation.



(a) Model B learning curves.



(b) Model B confusion matrix.

Figure 3: Model B training and test evaluation.

Table 6: Custom CNN test-set comparison.

Metric	Model A	Model B
Test accuracy	0.6959	0.6484
Macro precision	0.7057	0.6430
Macro recall	0.6648	0.6202
Macro F1-score	0.6750	0.6209
Parameters	101,829	14,272
FP32 parameter storage	397.77 KB	55.75 KB
Mean epoch time	12.8958 s	7.8730 s

Model A gives higher classification performance, reaching 0.6959 test accuracy and 0.7057 macro precision. Model B trades away some accuracy for a much smaller footprint: it uses only 14,272 parameters and trains faster per epoch. This makes Model B the stronger candidate when memory and compute are the primary constraints.

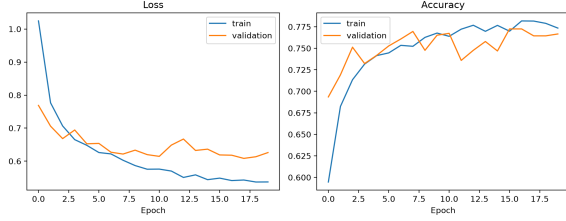
7 Pretrained Lightweight Models

Owner: Lavanathan.J – 230371R

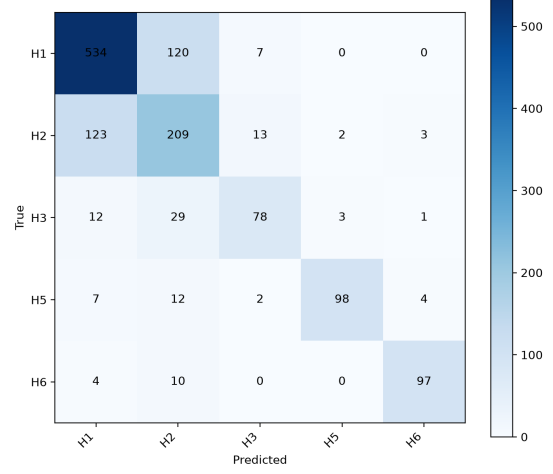
Two lightweight ImageNet-initialized architectures were evaluated using transfer learning. In both cases, the pretrained backbone was frozen and only a new classification head was trained. The head consisted of global average pooling, dropout with rate 0.2, and a five-unit Softmax classifier. No backbone fine-tuning or unfreezing was performed.

7.1 MobileNetV2

MobileNetV2 is designed for efficient visual recognition using inverted residual blocks and linear bottlenecks [2]. The model was trained with $224 \times 224 \times 3$ inputs and MobileNetV2 preprocessing.



(a) MobileNetV2 learning curves.



(b) MobileNetV2 confusion matrix.

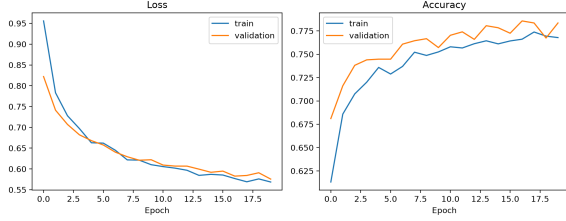
Figure 4: MobileNetV2 transfer-learning evaluation.

Table 7: MobileNetV2 results.

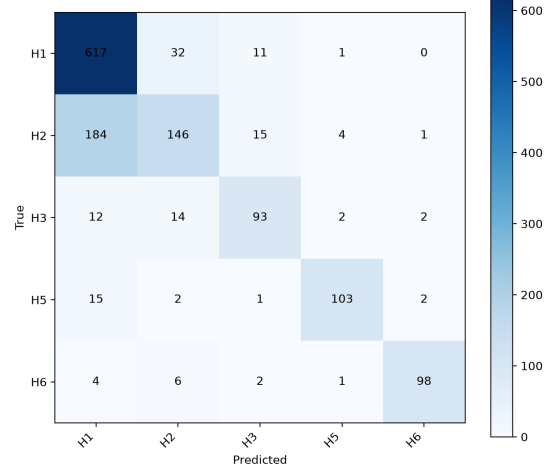
Metric	Value
Total parameters	2,264,389
Trainable parameters	6,405
Saved model size	9.25 MB
Mean epoch time	100.5135 s
Test accuracy	0.7427
Macro precision	0.7981
Macro recall	0.7420
Macro F1-score	0.7668

7.2 EfficientNetB0

EfficientNet uses compound scaling to balance network width, depth, and input resolution [3]. EfficientNetB0 was trained with $224 \times 224 \times 3$ inputs and EfficientNet preprocessing.



(a) EfficientNetB0 learning curves.



(b) EfficientNetB0 confusion matrix.

Figure 5: EfficientNetB0 transfer-learning evaluation.

Table 8: EfficientNetB0 results.

Metric	Value
Total parameters	4,055,976
Trainable parameters	6,405
Saved model size	16.33 MB
Mean epoch time	158.3490 s
Test accuracy	0.7727
Macro precision	0.8227
Macro recall	0.7654
Macro F1-score	0.7826

8 Final Model Comparison

Owner: Lavanathan.J – 230371R

The final comparison focuses on Model B because it is the custom resource-constrained model, and compares it against the two pretrained lightweight models. The computational-cost indicator is the measured mean epoch time in the CPU-only environment.

Table 9: Final comparison of the resource-constrained and pretrained models.

Metric	Model B	MobileNetV2	EfficientNetB0
Test accuracy	0.6484	0.7427	0.7727
Macro precision	0.6430	0.7981	0.8227
Macro recall	0.6202	0.7420	0.7654
Total parameters	14,272	2,264,389	4,055,976
Trainable parameters	14,272	6,405	6,405
Memory/model size	55.75 KB FP32 parameter storage	9.25 MB saved model	16.33 MB saved model
Computational-cost indicator	7.8730 s/epoch	100.5135 s/epoch	158.3490 s/epoch

EfficientNetB0 achieves the highest test accuracy, macro precision, and macro recall, but it also has the largest saved model size and the slowest measured epoch time. MobileNetV2 gives a strong accuracy improvement over Model B with a smaller saved model than EfficientNetB0. Model B is far smaller than both pretrained models and has the lowest measured training cost, but its accuracy is lower. Therefore, the preferred model depends on the deployment priority: Model B is most suitable when memory and compute are strict, while EfficientNetB0 is best when accuracy is the main goal and the larger model can be supported.

9 Conclusion

This project completed a resource-constrained image classification pipeline on the verified DeFungi dataset. The custom baseline Model A achieved 0.6959 test accuracy with 101,829 parameters, while the compact Model B reduced the parameter count to 14,272 and FP32 parameter storage to 55.75 KB, at a lower test accuracy of 0.6484. The optimizer comparison selected Adam with learning rate 0.001 based on the lowest final validation loss in the controlled optimizer experiment.

The pretrained models improved accuracy on the same test split. MobileNetV2 reached 0.7427 test accuracy, and EfficientNetB0 reached the best overall test accuracy of 0.7727. However, both pretrained models require substantially larger model storage and longer CPU epoch times than Model B. The final results show the expected tradeoff between compact custom models for edge deployment and pretrained models for stronger accuracy.

Table 10: Member contributions.

Member	Registration Number	Contribution
Rifnaz.K.R.M	230550P	Dataset Preparation + Model A
Panuharan.S	230462X	Model B + Efficiency Analysis
Peranavan.K	230474K	Optimizer Comparison + Training/Evaluation
Lavanathan.J	230371R	Pretrained Lightweight Models + Final Comparison

References

- [1] Farshid Hajati, Camilo Javier Pineda Sopo, Farshid Hajati, and Soheila Gheisari. DeFungi [dataset]. <https://archive.ics.uci.edu/dataset/773/defungi>, 2021. Accessed: 2026-10-02.
- [2] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4510–4520, 2018. doi: 10.1109/CVPR.2018.00474.
- [3] Mingxing Tan and Quoc Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR, 2019. URL <https://proceedings.mlr.press/v97/tan19a.html>.